Experiment 5

Aim: To implement shallow and deep parsing techniques for analyzing the grammatical structure of sentences.

Code:

Step 1: Check for the Python version

python --version

```
Python 3.11.0
```

Step 2: Download the important libraries

Download required NLTK resources

nltk.download('punkt')

nltk.download('punkt_tab')

nltk.download('averaged_perceptron_tagger_eng')

Output:

```
[nltk_data] Downloading package punkt to C:\Users\Mukesh
[nltk_data]     kushwaha\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to C:\Users\Mukesh
[nltk_data]     kushwaha\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     C:\Users\Mukesh kushwaha\AppData\Roaming\nltk_data...
[nltk_data]   Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data]       date!
True
```

Step 3: Import the required modules

import nltk

import spacy

from nltk import CFG

```
from nltk.parse import ChartParser
from spacy import displacy
```

Step 4: Define the input sentence

```
sentence = "The little boy eats an apple in the garden."
```

Step 5: Tokenization and POS tagging

```
tokens = nltk.word_tokenize(sentence)

print("TOKENS:")
print(tokens)

pos_tags = nltk.pos_tag(tokens)

print("\nPOS TAGS:")
for word, tag in pos_tags:
    print(word, "->", tag)
```

Output:

```
TOKENS:
['The', 'little', 'boy', 'eats', 'an', 'apple', 'in', 'the', 'garden', '.']

POS TAGS:
The -> DT
little -> JJ
boy -> NN
eats -> VBZ
an -> DT
apple -> NN
in -> IN
the -> DT
garden -> NN
. -> .
```

Step 6: Shallow Parsing

```
# Identify basic phrase-level structures
shallow_grammar = """
   NP: {<DT>?<JJ>*<NN>}
   PP: {<IN><NP>}
"""

shallow_parser = nltk.RegexpParser(shallow_grammar)

shallow_tree = shallow_parser.parse(pos_tags)

print("\nSHALLOW PARSING:")
print(shallow_tree)

print("\nSHALLOW PARSE TREE:")
shallow_tree.pretty_print()
```

Output:

```
SHALLOW PARSING:
(S
  (NP The/DT little/JJ boy/NN)
  eats/VBZ
  (NP an/DT apple/NN)
  (PP in/IN (NP the/DT garden/NN))
  ./.)

SHALLOW PARSE TREE:
                                S
    ____________________________|_____________________________
   |      |             |                  |                PP
   |      |             |                  |             ____|____
   |      |            NP                 NP            |         NP
   |      |     ________|_______        ___|_____        |     ____|______
eats/VBZ ./. The/DT little/JJ boy/NN an/DT     apple/NN in/IN the/DT     garden/NN
```

Step 7: Chunking

```
# Define rules for different types of chunks
chunk_grammar = """
   NP: {<DT>?<JJ>*<NN>}
   VP: {<VB.*><NP>}
   PP: {<IN><NP>}
"""

chunk_parser = nltk.RegexpParser(chunk_grammar)

chunk_tree = chunk_parser.parse(pos_tags)

print("\nCHUNKING:")
print(chunk_tree)

print("\nCHUNK TREE:")
chunk_tree.pretty_print()
```

Output:

```
CHUNKING:
(S
  (NP The/DT little/JJ boy/NN)
  (VP eats/VBZ (NP an/DT apple/NN))
  (PP in/IN (NP the/DT garden/NN))
  ./.)

CHUNK TREE:
                            S
  __________________________|_____________________________
 |             |                      VP                  PP
 |             |                  ____|____           _____|_____
 |            NP             |           NP         |           NP
 |     ________|_______       |        ___|_____     |     _____|______
./. The/DT little/JJ boy/NN eats/VBZ an/DT     apple/NN in/IN the/DT     garden/NN
```

Step 8: Deep parsing / Dependency Parsing

```
# Load spaCy English model
nlp = spacy.load("en_core_web_sm")

# Process the sentence
doc = nlp(sentence)

print("\nDEEP PARSING / DEPENDENCY PARSING:")

for token in doc:
    print(
        f"Word: {token.text}, "
        f"POS: {token.pos_}, "
        f"Tag: {token.tag_}, "
        f"Dependency: {token.dep_}, "
        f"Head: {token.head.text}"
    )
```

Output:

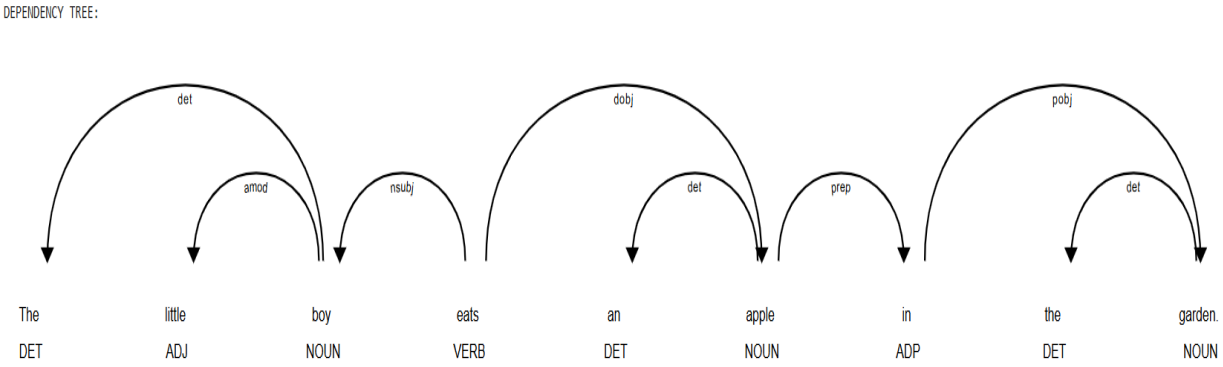
```
DEEP PARSING / DEPENDENCY PARSING:
Word: The, POS: DET, Tag: DT, Dependency: det, Head: boy
Word: little, POS: ADJ, Tag: JJ, Dependency: amod, Head: boy
Word: boy, POS: NOUN, Tag: NN, Dependency: nsubj, Head: eats
Word: eats, POS: VERB, Tag: VBZ, Dependency: ROOT, Head: eats
Word: an, POS: DET, Tag: DT, Dependency: det, Head: apple
Word: apple, POS: NOUN, Tag: NN, Dependency: dobj, Head: eats
Word: in, POS: ADP, Tag: IN, Dependency: prep, Head: apple
Word: the, POS: DET, Tag: DT, Dependency: det, Head: garden
Word: garden, POS: NOUN, Tag: NN, Dependency: pobj, Head: in
Word: ., POS: PUNCT, Tag: ., Dependency: punct, Head: eats
```

```
print("\nDEPENDENCY TREE:")

displacy.render(
```

```
    doc,
    style="dep",
    jupyter=True
)
```

Output:



Step 9: CFG based parse tree generation

```
cfg_grammar = CFG.fromstring("""
    S -> NP VP PP PUNC
    NP -> DT NN
    NP -> DT JJ NN
    VP -> VBZ NP
    PP -> IN NP

    DT -> 'The' | 'an' | 'the'
    JJ -> 'little'
    NN -> 'boy' | 'apple' | 'garden'
    VBZ -> 'eats'
```

```
    IN -> 'in'
    PUNC -> '.'
""")

parser = ChartParser(cfg_grammar)

cfg_tokens = nltk.word_tokenize(sentence)

print("\nCFG-BASED PARSE TREE:")

for tree in parser.parse(cfg_tokens):
    tree.draw()
```

Output:

